Load Testing Analysis ~ Java Spring Boot vs. Go Microservices

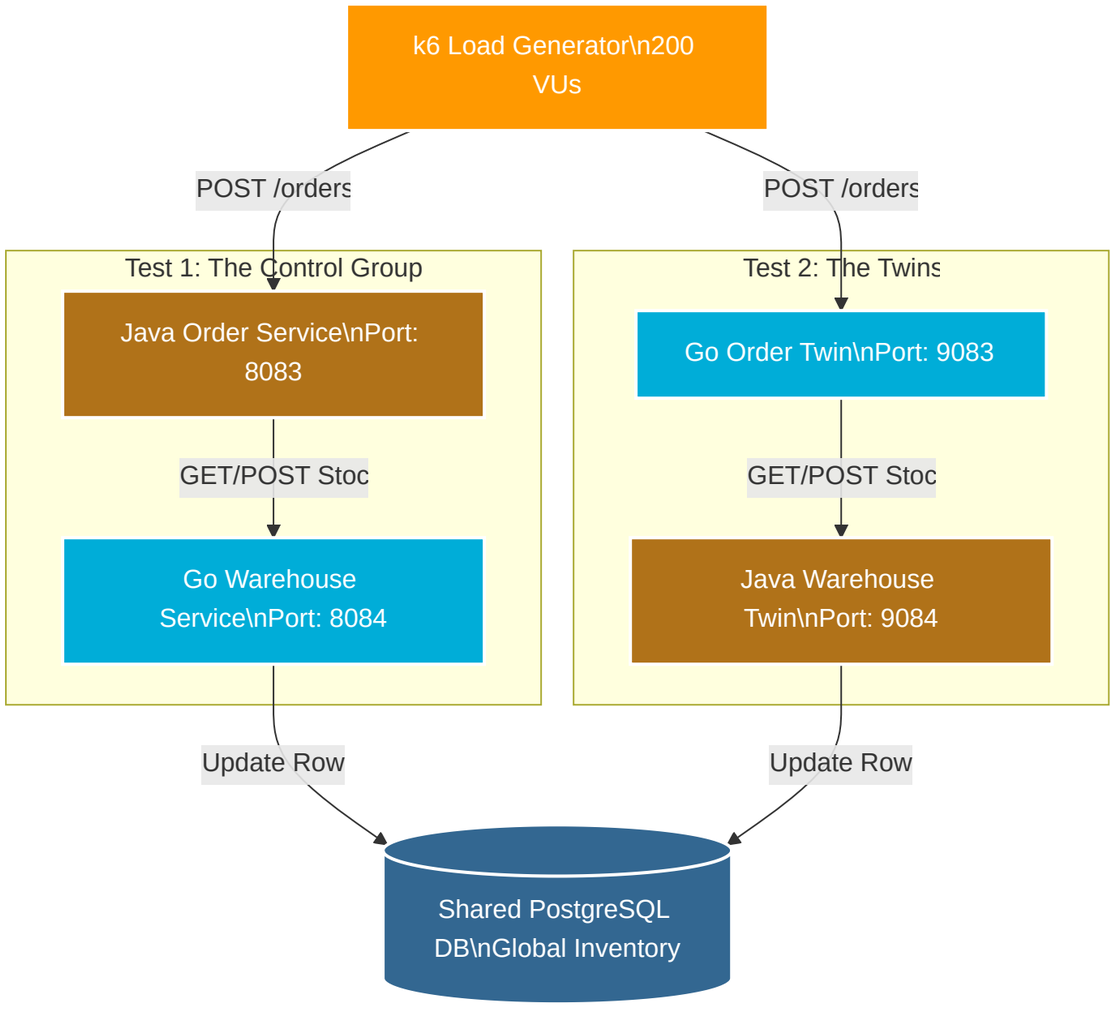
📝 Executive Summary

This document outlines the comparative runtime analysis of a legacy Java/Spring Boot microservice architecture against a newly developed Go "Twin" architecture. Load testing was conducted using `k6` simulating a spike of 200 concurrent Virtual Users (VUs).

The tests exposed critical bottlenecks related to **Database Row Locking**, **Connection Pool exhaustion**, **HTTP Client Timeouts**, and **ORM (Hibernate) Overhead**.

🏗️ Architecture & Traffic Flow

The system consists of two parallel processing chains interacting with a shared PostgreSQL database.



🔬 Phase 1: The Shared State Bottleneck & Error Masking

The Symptom

During initial tests, the Go Twin path (Test 2) failed rapidly with HTTP `409 Insufficient Stock` errors, despite the global inventory showing thousands of units available.

The Root Cause: Row-Level Locking & Timeouts

1. **The Traffic Jam:** 200 concurrent requests attempted to purchase the exact same `product_id`. To maintain ACID compliance, PostgreSQL applied a **Row-Level Lock** to that product's inventory row, forcing 200 requests to process serially.
2. **The Timeout:** As the queue grew, database updates took progressively longer (up to 600ms per row). The Go Order Twin's `http.Client` had a strict default timeout of **5 seconds**.
3. **Error Masking:** When the Go client gave up waiting for the Java Warehouse Twin, it threw a `context deadline exceeded` error. However, the Go code mapped *all* downstream errors to a generic `409 Insufficient Stock` response, hiding the true infrastructure bottleneck.

🐞 **Lesson Learned: Error Mapping Never map network timeouts (`504 Gateway Timeout` or `500 Internal Server Error`) to business logic failures (`409 Conflict`). It obscures infrastructure limits.**

⚙️ Phase 2: Unchoking the Java Warehouse

The Symptom

The Java Warehouse Twin (`9084`) was silently bottlenecking the Go Order Twin requests.

The Root Cause: Connection Pools

Spring Boot uses **HikariCP** for database connection pooling, which defaults to a maximum of 10 concurrent connections. Tomcat accepted 200 incoming HTTP threads, but 190 threads were forced to wait in a queue just to talk to the database.

✓ **Resolution Increased the Hikari pool size in `application.properties` to handle the spike: `spring.datasource.hikari.maximum-pool-size=200`**

🐢 Phase 3: ORM Overhead vs. Goroutines

The Symptom

After extending the Go Order Twin's HTTP timeout from 5 seconds to 30 seconds to allow the Java Warehouse Twin time to process the queue, the Go Twin *still* eventually timed out, with maximum request durations hitting an agonizing **38.71 seconds**.

The Root Cause: Hibernate Choke Point

While the Go Warehouse Service (Control Path) efficiently processed the row locks using lightweight goroutines and raw execution, the **Java Warehouse Twin** buckled under the concurrency.

The heavy overhead of Spring Data JPA and Hibernate—specifically setting up Transaction Boundaries, generating Entity Proxies, and performing Dirty Checking on 200 active threads—amplified the database lock wait times exponentially.

ⓘ **Architectural Philosophy: Fail Fast vs. Queue Forever The Go Twin's strict timeouts demonstrated a Fail Fast microservice pattern. In production, failing a request after 5 seconds is preferable to tying up server threads for 38 seconds, which can lead to cascading system failure.**

🚀 Phase 4: The Final Benchmark (Randomization)

To isolate the raw application speed and eliminate the PostgreSQL row-locking physics problem, the `k6` test script was rewritten to randomly select from an array of 3 fully-stocked products.

The Code Change (`order_benchmark.js`)

```
const PRODUCT_IDS = [
    'd2a2f90f-0af3-4e56-98bb-e7279bfc8a72', // iPhone 15 Pro
    '524a62c0-43ee-49a3-9dda-2a80398cdeba', // Sony WH-1000XM5
    'c193ad20-b449-4698-8ac0-3ee7365e805c'  // Samsung S24 Ultra
];
const randomProductId = PRODUCT_IDS[Math.floor(Math.random() * PRODUCT_IDS.length)];
```

The Final Outcomes

Metric	Test 1: Control (Java -> Go)	Test 2: Twins (Go -> Java)
Success Rate	99.87%	64.26%
Median Request Time	~16.8 seconds	~21.2 seconds
Max Request Time	25.8 seconds	38.7 seconds
Primary Bottleneck	Initial JVM Cold Start	ORM/Hibernate Thread Overhead

Final Conclusion

- **Functional Parity Achieved:** The Go to Spring Boot conversions execute the exact same business logic and smart routing.
- **Performance Divergence:** Under extreme, highly-concurrent database contention, the statically-linked Go microservices (with raw/lightweight DB drivers) significantly outperform the JIT-compiled Spring Boot microservices relying on heavy ORM frameworks.